ConcurrentCollections:->

BlockingQueueExample (HIGH):->

```
BlockingQueueExample.java:->
```

```java
package com.example.demo;


import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

public class BlockingQueueExample {
    public static void main(String[] args) {
        BlockingQueue<Integer>queue=new LinkedBlockingQueue<>();
        Thread producer=new Thread(()->produce(queue));
        Thread consumer=new Thread(()->consume(queue));
        producer.start();
        consumer.start();
        try{
            producer.join();
            consumer.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    private static void produce(BlockingQueue<Integer> queue) {
        for(int i=1;i<=5;i++)
        {
            try {
                queue.put(i);
                System.out.println("Produced: "+i);
                Thread.sleep(500L);
            } catch (InterruptedException e) {
                throw new RuntimeException();
            }
        }

    }
    private static void consume(BlockingQueue<Integer> queue) {
        for(int i=1;i<=5;i++)
        {
            try{
                int value=queue.take();
                System.out.println("Consumed: "+value);
                Thread.sleep(500L);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
    }
}
```

ThreadSafeCollectionsExample.java:->

```java
package com.example.demo;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class ThreadSafeCollectionsExample {
    public static void main(String[] args) {
        List<String>orderList=new CopyOnWriteArrayList<>();
        Thread t1=new Thread(()->addOrder(orderList,"Order 1"));
        Thread t2=new Thread(()->addOrder(orderList,"Order 2"));
        t1.start();
        t2.start();
        try{
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Final order list: "+orderList);
    }

    private static void addOrder(List<String> orderList, String order) {
        orderList.add(order);
        System.out.println(Thread.currentThread().getName()+" added:
"+order);
    }

}
```